OntoLoop: A Governed, Replayable Runtime for Sovereign Language-Model Agents

AutoLoop Contributors
`anomalyco/autoloop-app`
Draft systems paper

May 12, 2026

Abstract

Language-model agents are increasingly used as tool-using software operators: they plan, call external systems, modify artifacts, learn from feedback, and continue work across sessions. Existing agent frameworks have demonstrated strong reasoning, tool-use, and self-improvement capabilities, but production use introduces a different class of requirements: policy enforcement, tenant isolation, budget control, replayability, rollback, and auditable evidence for every consequential action. This paper presents OntoLoop, an engineering runtime for governed and replayable language-model agents. OntoLoop decomposes agent execution into a policy-control layer, planning layer, guarded execution layer, verification layer, learning layer, and reporting layer. The runtime centers all provider and tool activity around typed task envelopes, tri-state admission decisions, append-only evidence records, deterministic replay boundaries, and release gates that block production writes when evidence is incomplete. We describe the architecture, contracts, implementation, and evaluation protocol of the current open-source alpha artifact, implemented primarily in Rust with a Vue control plane and local CLI surfaces. Rather than optimizing only for task success, OntoLoop treats agent autonomy as a systems problem: every successful action must be attributable, bounded, recoverable, and explainable.

1 Introduction

Large language models (LLMs) have moved from passive text generation toward interactive agents that reason, call tools, inspect environments, write code, and adapt over time. Work such as ReAct [1], Toolformer [2], Reflexion [3], Tree of Thoughts [4], Voyager [5], and SWE-agent [8] shows that combining models with tools, feedback, memory, and iterative control can greatly expand practical capability. However, capability alone is not a runtime contract. A production agent must also answer operational questions:

- Which identity, tenant, policy, and budget authorized an action?
- Which tool or provider path was selected, and was it verified at the time?
- What evidence proves that an artifact was actually written, hashed, and linked to the corresponding decision?
- Can an operator replay a session, explain drift, and roll back a failed rollout?
- Can tests and static checks prevent direct bypass of the runtime guard?

OntoLoop is designed around these questions. We use *sovereign runtime* in an engineering sense: the local system, not the model provider, owns policy, admission, budget, evidence, replay, and rollback boundaries. The objective is not to replace model-level alignment methods such as Constitutional AI [6], but to complement them with deterministic software contracts around the agent's actions.

The current OntoLoop repository is an alpha engineering artifact. Its Rust workspace contains a primary runtime crate, state and PostgreSQL adapters, an immutable execution constitution crate, and a trust-kernel crate. The control surfaces include a product CLI, a Node-based wrapper, and a Vue dashboard for snapshots, replay, capability governance, and operator settings. The repository also contains layered acceptance scripts and integration tests that encode the runtime's hard gates.

This paper makes four contributions:

1. A layered architecture for governed LLM-agent execution that separates policy control, planning, execution, verification, learning, and reporting.

2. A runtime contract centered on typed task envelopes, tri-state admission, no-bypass enforcement, evidence-linked artifact delivery, and replayable decision records.

3. An implementation blueprint for integrating providers, tools, memory, GraphRAG-style knowledge, triggers, session replay, and operator dashboards without allowing direct execution paths around the guard.

4. A release-oriented evaluation protocol that treats acceptance gates, rollback evidence, and replay consistency as first-class measures of agent readiness.

2 Design Goals

OntoLoop is built for agent workloads that may touch files, tools, external services, memory stores, and business-facing control planes. The design is guided by five goals.

G1: All execution passes through one accountable gate. Provider calls, tool calls, MCP-style capability invocations, and artifact writes must be mediated by the runtime kernel. The agent should not be able to call a high-impact path through an untracked shortcut.

G2: Policy decisions are explicit and typed. Admission returns one of three states: `Allow`, `RequiresApproval`, or `Blocked`. Every decision carries a reason and evidence linkage.

G3: Evidence is append-only and replayable. Execution produces durable records that bind session identifiers, trace identifiers, task identifiers, capability identifiers, policy versions, artifact hashes, and decision outcomes.

G4: Learning is governed. Feedback, memory updates, skill promotion, and route optimization are treated as runtime events subject to verification and rollback, not as invisible prompt drift.

G5: Release readiness is contract-driven. OntoLoop does not define production readiness as passing a single benchmark. It requires layered checks: preflight, contract, domain suites, full-chain acceptance, and release gate fusion.

3 Architecture

Figure 1 shows the canonical execution path as a text diagram. A user intent or trigger enters the transport/session layer, is compiled into a structured turn state, passes through planning and policy context, executes through guarded capability routing, and finally emits evidence, learning updates, replay artifacts, and observability reports.

```
User/Trigger
  -> Structured Transport + Session Bridge
  -> Query Turn State Machine + Context Compiler/Compactor
  -> Requirement Clarification + Constraint Freeze
  -> Policy Context + Knowledge Context
  -> Orchestrator + Capability Router
  -> Capability Admission
  -> Trust Admission Kernel
  -> Runtime Guard + Permission Mode
  -> Layered Tool Execution + Execution Fabric
  -> Evidence Tagger + Flow State Engine
  -> Trust Evidence Ledger
  -> Verifier + Audit
  -> Learning + Promotion Gate
  -> Memory / Org Knowledge Update
  -> Unified Query / Replay / Explanation
  -> Observability + Reports
```

Figure 1: Condensed OntoLoop runtime flow.

3.1 Layer Model

OntoLoop maps high-level agent behavior onto neutral software layers:

Layer	Role	Primary modules
Policy control	Policy, risk, budget, immutable constraints	`security`, `runtime`, `config`
Planning	Clarification, scope freeze, decomposition	`orchestration`, `agent`
Execution	Capability selection, tool/provider invocation	`tools`, `providers`, `runtime`
Resource kernel	CPU, memory, time, queue, sandbox control	`runtime`, `security`
Verification	Result validation, policy checks, reject loops	`runtime`, `observability`
Learning	Evidence, skills, causal edges, route memory	`memory`, `rag`, `hooks`
Reporting	Snapshot, replay, dashboard, health telemetry	`observability`, `dashboard-ui`

Table 1: OntoLoop's neutral layer decomposition.

3.2 Runtime Core

The main Rust application object assembles configuration, providers, tools, security policy, memory, RAG, sessions, transport, plugins, services, observability, and the runtime kernel. Its CLI exposes command families for focus anchors, knowledge export, triggers, bridges, skills, plugins, sessions, background tasks, memory, frontend status, and system reports.

The runtime kernel owns:

- resource limits and parallelism windows;
- MCP/tool circuit breaker thresholds and cooldowns;
- gate mode (`shadow`, `canary`, or `full`);
- permission-mode defaults and policy-mode defaults;
- budget and quota enforcement;
- attestation configuration;
- hook runtime and layered tool execution.

3.3 Control Plane

The dashboard exposes local operational APIs for health, dashboard snapshots, replay timelines, capability catalog views, capability governance, operator settings, trigger webhooks, and server-sent event streams. The Vue control plane renders summary strips, runtime sidebars, graph canvases, capability tables, detail workbenches, replay timelines, and governance panels.

4 Contracts

OntoLoop's central mechanism is the conversion of agent behavior into explicit contracts. The contract layer is intentionally more restrictive than a typical agent loop because the system must preserve auditability under failure.

4.1 Task Envelope

Every consequential execution is represented as a task envelope:

```
TaskEnvelope {
  session_id,
  trace_id,
  task_id,
  capability_id,
  identity,
  payload,
  constraints,
  trust_plan
}
```

The envelope binds an action to identity, constraints, and trace context before the runtime can execute it. This gives downstream evidence records stable keys.

4.2 Admission

Admission decisions are tri-state:

$$d \in \{\texttt{Allow}, \texttt{RequiresApproval}, \texttt{Blocked}\}.$$

The runtime does not treat "not blocked" as sufficient. A path that requires approval must remain distinct from a path that is automatically allowed. This distinction is important for human-in-the-loop operations, gray rollout, and post-incident forensics.

4.3 Production Write Contract

For production writes, OntoLoop uses a five-part atomic contract:

$$W = (\texttt{state}, \texttt{event\_log}, \texttt{evidence\_ref}, \texttt{relation}, \texttt{write\_proof}).$$

A write is valid only when all five elements commit together or the transaction rolls back. This prevents common failure modes where a state mutation exists without evidence, or an event log claims an artifact that was never written.

4.4 Artifact Delivery

An artifact-completion claim is invalid without a write proof, hash, and evidence reference. This is especially important for coding agents because natural-language answers can claim completion even when no file-backed artifact exists.

4.5 NoBypass

NoBypass combines three enforcement layers:

1. static scans for direct provider/tool/memory/MCP shortcuts;
2. compile-time gates around contract boundaries;
3. runtime gates around every task envelope.

This pattern aims to catch bypass attempts before they become runtime incidents.

5 Lifecycle

The canonical lifecycle is:

1. `INTENT_INTAKE`: normalize input into a requirement brief.
2. `CONSTRAINT_FREEZE`: freeze acceptance criteria, risk profile, budget, and hard boundaries.
3. `PLAN_SYNTHESIS`: generate plans, critiques, and an executable route.
4. `CAPABILITY_ROUTING`: select only active and verified capabilities.
5. `GUARDED_EXECUTION`: enforce budgets, timeout, sandbox, and circuit breaker policy.
6. `VERIFICATION_GATE`: pass, iterate, or reject.

7. `LEARNING_COMMIT`: persist episodes, witness logs, skill updates, and graph deltas.
8. `OBSERVABILITY_EMIT`: publish snapshots, replay events, and reports.

The state machine supports recovery edges. A verifier rejection returns the task to planning; a circuit breaker or budget failure returns execution to planning; policy denial moves the task to a blocked state.

6 Memory and Knowledge

OntoLoop includes a memory subsystem for reflexion episodes, witness logs, skill records, causal edges, and SuperMemory-style pipelines. It also includes GraphRAG components for entity-relation extraction, graph updates, retrieval, sharing gates, and organization knowledge publication. The design is related to GraphRAG [7] in that graph structure is used to support global context and sensemaking, but OntoLoop focuses on operational traceability: memory updates are linked to evidence and promotion gates.

Learning is therefore not simply prompt accumulation. A learning delta must be derived from evidence, scored, verified, and optionally promoted. This prevents the agent from silently converting one unverified failure into future policy.

7 Implementation

The current artifact is implemented as a Rust workspace with auxiliary frontend and deployment surfaces. The inspected repository contains:

- 215 Rust source files under `src/`;
- 85 Rust integration test files under `tests/`;
- 63 acceptance and operations scripts under `deploy/scripts/`;
- 72 frontend source files under `dashboard-ui/src/`.

The workspace members are:

- `ontoloop`: the primary runtime and CLI crate;
- `autoloop-state-adapter`: internal StateStore-compatible state adapter with in-memory and PostgreSQL bridge paths;
- `autoloop-postgres-adapter`: PostgreSQL-backed state and evidence adapter;
- `trustkernel`: a verified execution kernel crate housed under `NoumenonCore`;
- `ontoloop-core`: immutable execution constitution contracts.

The frontend control plane is a Vue 3 and Vite application. The Node package wrapper exposes an `ontoloop` binary that locates the Rust executable and launches either direct message mode, subcommand passthrough, or TUI mode.

8 Evaluation Protocol

OntoLoop's evaluation protocol is release-oriented. A model-only benchmark can measure whether an agent solves a task; OntoLoop additionally asks whether the solution is authorized, bounded, attributed, replayable, and recoverable.

8.1 Layered Acceptance

The acceptance model contains five layers:

Layer	Scope	Blocking condition
L0	profile, config, storage readiness	failed preflight blocks all later layers
L1	WAL, admission, evidence, NoBypass	any contract breach blocks release
L2	sandbox, frontend CLI, storage, signals	any required domain failure blocks L3/L4
L3	integrated full-chain acceptance	`all_passed != true` blocks release
L4	fused release decision	`allow_release=false` blocks rollout

Table 2: Layered acceptance protocol.

8.2 Suggested Metrics

For future empirical submissions, we recommend reporting:

- task success rate under guarded execution;
- percentage of actions with complete evidence records;
- replay match rate and explainable drift rate;
- NoBypass static/compile/runtime coverage;
- rollback latency after canary failure;
- budget reconciliation accuracy;
- tenant isolation violations;
- operator intervention rate and approval latency.

These metrics should be reported by release gate artifacts rather than prose-only claims. The current paper intentionally does not invent benchmark numbers for the alpha artifact.

9 Related Work

Reasoning and tool-use agents. ReAct interleaves reasoning traces and actions [1], while Toolformer studies self-supervised tool-use learning [2]. OntoLoop assumes such tool-using behavior is useful, but wraps it in explicit authorization and evidence contracts.

Search, reflection, and lifelong learning. Tree of Thoughts expands inference into deliberate search over intermediate thoughts [4]. Reflexion stores verbal feedback in episodic memory [3]. Voyager demonstrates open-ended skill acquisition in an embodied environment [5]. OntoLoop's learning layer is closest in spirit to these systems, but it treats skill promotion as a governed runtime event rather than an unconstrained memory update.

Software-engineering agents. SWE-agent shows that agent-computer interfaces matter for automated software engineering [8]. OntoLoop adds contract and evidence layers around the computer interface: artifact claims require write proofs, and execution must pass the runtime guard.

Alignment and policy. Constitutional AI uses rule-based critique and AI feedback for model behavior control [6]. OntoLoop operates at the systems layer: policies are evaluated as runtime gates over concrete tasks, identities, tools, budgets, and artifacts.

Graph-based retrieval. GraphRAG builds graph indexes and community summaries to improve global query-focused summarization [7]. OntoLoop uses graph memory for operational knowledge and routing evidence, with emphasis on provenance and promotion gates.

10 Limitations

OntoLoop is currently an alpha engineering runtime. Several limitations remain. First, the artifact's production hardening is uneven across provider and MCP surfaces. Second, rich real-time replay visualization is not yet complete. Third, the dashboard and policy administration interfaces are local-operation oriented, not a full enterprise identity product. Fourth, quantitative benchmark results must be generated by the release scripts and checked into an evidence package before this paper can be treated as a results paper rather than a systems design paper. Finally, the current repository has substantial ongoing changes; release claims should be tied to clean commits and immutable artifacts.

11 Conclusion

OntoLoop frames LLM-agent autonomy as a runtime systems problem. The central idea is simple: an agent action should not count as complete merely because a model said so or a tool returned text. It must be authorized, bounded, executed through a guarded path, linked to evidence, replayable, and recoverable. The current alpha artifact implements this idea through Rust runtime contracts, typed task envelopes, tri-state admission, evidence ledgers, no-bypass checks, session replay, governed learning, and layered release gates. This design points toward a class of agentic systems where autonomy and accountability are engineered together rather than traded against each other.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022. `https://arxiv.org/abs/2210.03629`

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*, 2023. https://arxiv.org/abs/2302.04761

[3] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*, 2023. https://arxiv.org/abs/2303.11366

[4] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *arXiv preprint arXiv:2305.10601*, 2023. https://arxiv.org/abs/2305.10601

[5] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023. https://arxiv.org/abs/2305.16291

[6] Yuntao Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*, 2022. https://arxiv.org/abs/2212.08073

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*, 2024. https://arxiv.org/abs/2404.16130

[8] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024. https://arxiv.org/abs/2405.15793